

Polymorphism

林致宇

Review

- Object Oriented
 - Encapsulation / Abstraction / Inheritance / Polymorphism
封 裝 抽 象 繼 承 多 形

Outline

- Introduction to Polymorphism
- Virtual Function
- Pure Virtual Functions and Abstract Classes

Outline

- Introduction to Polymorphism
- Virtual Function
- Pure Virtual Functions and Abstract Classes

Types of Polymorphism

編譯時期的多形

- Compile Time Polymorphism
 - Function Overloading 函式多載
 - Operator Overloading 運算子多載

Runtime 時期的多形

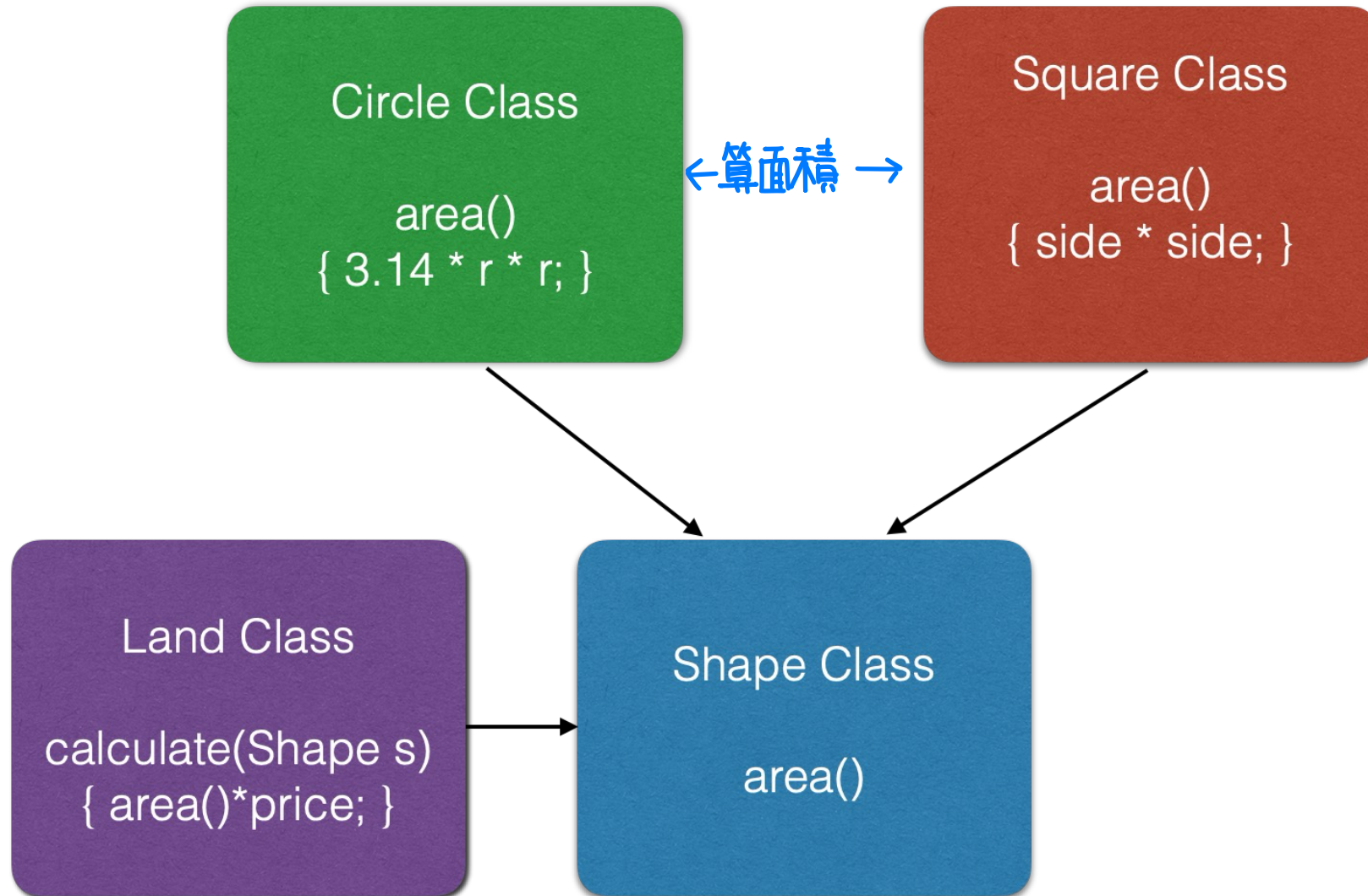
- **Runtime Polymorphism** ← 基於繼承而來的
 - **Virtual Function**

Polymorphism

父類別的物件指標

- Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.
- The process of representing one form in multiple forms is known as Polymorphism. Here one form represent original form or original method always resides in base class and multiple forms represents overridden method which resides in derived classes.

Polymorphism



Parent Class Reference

```
4 class Shape {
5 public:
6     int x = 3;
7     void showX() {
8         cout << "(Shape) " << x << endl;
9     }
10 };
11
12 class Circle: public Shape {
13 public:
14     int x = 5;
15     void showX() {
16         cout << "(Circle) " << x << endl;
17     }
18 };
19
20 int main(int argc, const char * argv[]) {
21     Circle *c = new Circle;
22     c->showX();
23     Shape *s = new Circle;
24     s->showX();
25 }
```

如果要讓 Line 24 Call 子類別的 showX:

多形! →

↑ 父類別的指標, 指到子類別的物件? → Call 父類別的 showX

一個 child class object

会考!

```
lincyu@Chih-YudeMacBook-Pro Chapter13 % ./a.out
(Circle) 5
(Shape) 3
```

Casting

```
4  class Shape {
5  public:
6      int x = 3;
7      void showX() {
8          cout << "(Shape) " << x << endl;
9      }
10 };
11 class Circle: public Shape {
12 public:
13     int x = 5;
14     void showX() {
15         cout << "(Circle) " << x << endl;
16     }
17 };
18 int main(int argc, const char * argv[]) {
19     Circle *c = new Circle;
20     c->showX();
21     Shape *s = new Circle;
22     ((Circle *)s)->showX();
23 }
```

⚠ 此操作有点危險!

將 s 強制轉形成子類別的指標
也可以達成 call 子類別的 showX

Outline

- Introduction to Polymorphism
- Virtual Function
- Pure Virtual Functions and Abstract Classes

Virtual Function

- A virtual function is a member function which is declared **within a base class** and is **re-defined (overridden) by a derived class**. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute **the derived class's version** of the function.
- Reference: <https://www.geeksforgeeks.org/virtual-function-cpp/>

Virtual Function

会考!

```
4  class Shape {
5  public:  ↙ 沒有 virtual, 会以父的function為主
6          virtual double area();
7  };
8  double Shape::area() { return 0; }
9
10 class Circle: public Shape {
11 public:
12     double radius;
13     double area() {
14         return 3.14 * radius * radius;
15     }
16 };
17 class Squire: public Shape {
18 public:
19     double side;
20     double area() { return side * side; }
21 };
```

```
23 double getTotalPrice(Shape *s) {
24     return s->area()*2.0;
25 }
26
27 int main(int argc, const char * argv[]) {
28     Circle c;
29     c.radius = 2.0;
30     cout << getTotalPrice(&c) << endl;;
31     Squire s;
32     s.side = 2.0;
33     cout << getTotalPrice(&s) << endl;;
34 }
```

因為Line 6 有 virtual
所以呼叫的是子類別的 area

Outline

- Introduction to Polymorphism
- Virtual Function
- Pure Virtual Functions and Abstract Classes
(Abstract Function)

Pure Virtual Function

C++ 的 pure virtual function 為只初始化，
但沒有定義的函式。

A pure virtual function in c++ is defined as a function that is only initialized but not defined. A pure virtual function in c++ must end with "=0" when declared. → 要以 "=0" 作結尾

```
4 class Shape {
5 public:
6     virtual double area() = 0;
7 };
8
9 class Circle: public Shape {
10 public:
11 };
12
13 int main(int argc, const char * argv[]) {
14     Circle c;
15 }
```

Pure Virtual Function

一定要去 override area,
不然也會是 Abstract Class

```
lincyu@Chih-YudeMacBook-Pro Chapter13 % g++ 13_03_PureVirtualFunction1.cpp
13_03_PureVirtualFunction1.cpp:14:12: error: variable type 'Circle' is an
abstract class
    Circle c;
    ^
13_03_PureVirtualFunction1.cpp:6:20: note: unimplemented pure virtual meth
od 'area' in 'Circle'
    virtual double area() = 0;
                   ^
1 error generated.
```

有 Pure Virtual Function 的類別就是
抽象類別 (Abstract class)，Abstract
Class 自己是不能產生物件的。

Pure Virtual Function

```
4  class Shape {
5  public:
6      virtual double area() = 0;
7  };
8
9  class Circle: public Shape {
10 public:
11     double radius;
12     double area() { return 3.14 * radius * radius; }
13 };  Override area
14
15 int main(int argc, const char * argv[]) {
16     Circle c;
17 }
```